BTS CIEL · PROGRAMMATION PHP · TRAVAUX PRATIQUES

TP1 — Carnet de contacts

Cours associé : `01_Cours/Cours_PHP.pdf`.

Objectif

Créer une page web capable d'afficher des contacts, de traiter et valider les données d'un formulaire, puis de conserver les contacts ajoutés.

Prérequis : sections 5, 7, 9, 10 et 11 du cours ; section 8 (expressions régulières) pour la partie C « pour aller plus loin » ; section 12 (PDO) pour la partie G.

Partie A — Afficher des contacts

On fournit le code suivant :

```php
<?php
$contacts = [
    ["nom" => "Martin",  "email" => "martin@example.com",  "tel" => "0612345678"],
    ["nom" => "Dupont",  "email" => "dupont@example.com",  "tel" => "0623456789"],
    ["nom" => "Bernard", "email" => "bernard@example.com", "tel" => "0634567890"],
];
?>

<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Mon carnet de contacts</title>
</head>
<body>
    <h1>Mon carnet de contacts</h1>

    <?php foreach ($contacts as $contact) : ?>
        <p>
            <?= htmlspecialchars($contact["nom"], ENT_QUOTES, "UTF-8") ?>
            —
            <?= htmlspecialchars($contact["email"], ENT_QUOTES, "UTF-8") ?>
            —
            <?= htmlspecialchars($contact["tel"], ENT_QUOTES, "UTF-8") ?>
        </p>
    <?php endforeach; ?>
</body>
</html>
```

Travail demandé :

1. Enregistrer le fichier sous `contacts.php`.
2. Afficher la page dans le navigateur.
3. Ajouter un quatrième contact.
4. Modifier le nom d'un contact existant.
5. Repérer la boucle `foreach` et expliquer à quoi elle sert.

Partie B — Premier traitement de formulaire

On fournit le formulaire HTML :

```html
<form method="post">
    <label for="nom">Nom :</label>
    <input type="text" id="nom" name="nom" required>

    <button type="submit">Envoyer</button>
</form>
```

Vous devez compléter le code PHP :

```php
<?php
$nom = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $nom = trim($_POST["nom"] ?? "");
}
?>

<?php if ($nom !== "") : ?>
    <p>
        Bonjour <?= htmlspecialchars($nom, ENT_QUOTES, "UTF-8") ?> !
    </p>
<?php endif; ?>
```

Travail demandé :

1. Tester le formulaire avec un nom.
2. Expliquer le rôle de `method="post"`.
3. Repérer la condition qui détecte l'envoi du formulaire.
4. Expliquer pourquoi on utilise `htmlspecialchars()`. Tester la saisie `<b>Martin</b>` en retirant temporairement la fonction.
5. Tester la saisie avec des espaces avant et après le nom : quel est le rôle de `trim()` ?

Partie C — Formulaire complet et validation

Remplacer le formulaire de la partie B par un formulaire d'ajout de contact à **trois champs** : `nom`, `email` et `telephone`.

```html
<form method="post" novalidate>
    <label for="nom">Nom :</label>
    <input type="text" id="nom" name="nom">

    <label for="email">Email :</label>
    <input type="text" id="email" name="email">

    <label for="telephone">Téléphone :</label>
    <input type="text" id="telephone" name="telephone">

    <button type="submit">Ajouter</button>
</form>
```

> `novalidate` désactive la validation du navigateur pour tester la validation PHP. On pourra le retirer à la fin.

Règles à vérifier **côté serveur** :

- le nom ne doit pas être vide ;
- l'adresse électronique doit passer la validation PHP ;
- le téléphone ne doit contenir que des chiffres.

Exemple de validation à intégrer dans le traitement du formulaire :

```php
$erreurs = [];

$nom       = trim($_POST["nom"] ?? "");
$email     = trim($_POST["email"] ?? "");
$telephone = trim($_POST["telephone"] ?? "");

if ($nom === "") {
    $erreurs[] = "Le nom est obligatoire.";
}

if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $erreurs[] = "L'adresse email est invalide.";
}

if ($telephone === "" || !ctype_digit($telephone)) {
    $erreurs[] = "Le téléphone doit contenir uniquement des chiffres.";
}
```

Ce code suppose que le traitement est exécuté après l'envoi du formulaire : les champs doivent être initialisés et le traitement doit être protégé par une condition vérifiant la méthode de requête (`$_SERVER["REQUEST_METHOD"] === "POST"`).

Travail demandé :

1. Intégrer ce code dans la page et afficher les erreurs **une par une**, en échappant leur contenu lors de l'affichage HTML.
2. Si la liste d'erreurs est vide, ajouter le contact au tableau `$contacts` et afficher un message de confirmation.
3. En cas d'erreur, conserver dans le formulaire les valeurs déjà saisies.
4. *Pour aller plus loin :* `ctype_digit()` refuse « 06 12 34 56 78 ». Accepter les espaces avec `preg_match('/^0\d(?: ?\d{2}){4}$/', $telephone)`.

Partie D — Affichage des données sous forme d'un tableau

```php
<?php if (empty($contacts)) : ?>
    <p>Aucun contact pour le moment.</p>
<?php else : ?>

    <table border="1" cellpadding="5">
        <tr>
            <th>Nom</th>
            <th>Email</th>
            <th>Téléphone</th>
        </tr>

        <?php foreach ($contacts as $c) : ?>
            <tr>
                <td><?= htmlspecialchars($c["nom"], ENT_QUOTES, "UTF-8") ?></td>
                <td><?= htmlspecialchars($c["email"], ENT_QUOTES, "UTF-8") ?></td>
                <td><?= htmlspecialchars($c["tel"], ENT_QUOTES, "UTF-8") ?></td>
            </tr>
        <?php endforeach; ?>
    </table>

<?php endif; ?>
```

Travail demandé :

1. Remplacer l'affichage en paragraphes de la partie A par ce tableau.
2. Ajouter un contact valide : il apparaît dans le tableau. **Recharger la page** : que devient-il ? Pourquoi ?
3. Pourquoi chaque valeur est-elle passée dans `htmlspecialchars()`, y compris dans le tableau ?

Partie E — Étape optionnelle : `header.php` / `footer.php`

`header.php`

```php
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title><?= htmlspecialchars($titre ?? "Carnet de contacts", ENT_QUOTES, "UTF-8") ?></
title>
</head>
<body>
```

`footer.php`

```php
</body>
</html>
```

Dans chaque page principale :

```php
<?php
$titre = "Mes contacts";
include "header.php";
?>

... contenu ...

<?php include "footer.php"; ?>
```

Partie F — Conserver les contacts avec la session

Les contacts ajoutés disparaissent à chaque requête. On les conserve dans `$_SESSION`.

1. Appeler `session_start()` tout en haut de la page.
2. Au premier chargement, initialiser `$_SESSION["contacts"]` avec les trois contacts de départ (`$_SESSION["contacts"] ??= [...]`).
3. Ajouter chaque contact valide à `$_SESSION["contacts"]` et afficher ce tableau.
4. Ajouter un bouton **Supprimer** par ligne : un petit formulaire POST contenant l'indice du contact en champ caché, traité avec `unset()` (vérifier que l'indice est un entier existant).
5. Quelle est la limite de cette solution ? (indice : fermer le navigateur, changer de poste.)

Partie G — Bonus : stocker les contacts dans MariaDB

Créer la table `contacts` (`id`, `nom`, `email`, `telephone`) puis remplacer la session par des requêtes **préparées** PDO (section 12) : `INSERT` pour l'ajout, `SELECT` pour l'affichage, `DELETE` pour la suppression.